

1 Постановка задачі та вимоги до програми

1.1 Стан питання та обґрунтування теми

Об'єктом дослідження є процес автоматичного групування природномовних текстів. Предмет дослідження становлять методи попередньої обробки україномовних документів, їх векторного подання та кластеризації, а також співвідношення якості групування й обчислювальних витрат. Практичне завдання полягає у створенні локального інструмента, який дає змогу проводити експерименти за єдиною схемою, зіставляти їхні результати та зберігати дані для подальшого аналізу.

У текстових колекціях одну тему можна описати різними словами, а те саме слово може мати кілька значень. Для української мови додаткову складність створює словозміна: без нормалізації різні форми одного слова збільшують словник ознак. Тому якість кластеризації залежить як від алгоритму групування, так і від очищення тексту, лематизації та способу формування векторів. Доцільність поєднання цих етапів необхідно перевіряти експериментально.

TF-IDF є базовим методом векторизації, результати якого легко інтерпретувати, проте тексти спільної тематики не завжди містять однакові слова. Word2Vec формує вектори слів у процесі навчання, а FastText додатково враховує частини слова [3; 4]. Doc2Vec формує подання документів [5]. Sentence-BERT створює подання речень для семантичного порівняння [6]. Кожен підхід враховує певні властивості тексту, тому вибір методу для україномовного корпусу потребує перевірки на відповідних даних.

Наявні програмні засоби підтримують значну частину необхідних операцій. Бібліотека scikit-learn надає алгоритми кластеризації та оцінювання, Gensim — моделі векторизації, а Orange — засоби візуального побудування конвеєрів обробки тексту [2–4; 7]. Внесок роботи полягає в інтеграції цих типів компонентів у спеціалізований дослідницький інструмент і перевірці власної реалізації паралельних перезапусків K-Means.

1.2 Невирішене питання та мета роботи

Для конкретного корпусу залишається відкритим питання, яка комбінація обробки, векторизації та кластеризації забезпечує прийнятне тематичне групування за доступних ресурсів. Мета роботи — розробити й перевірити інструментальні засоби для такого порівняння. Завдання охоплюють імпорт даних, уніфікацію конвеєра, реалізацію Parallel K-Means, розрахунок метрик, вимірювання ресурсів і формування звітів. Перевага вдосконаленого підходу розглядається як гіпотеза, що потребує перевірки.

1.3 Критичне зіставлення підходів

Відмінності між оцінками методів не обов'язково свідчать про суперечність результатів. Високе значення коефіцієнта силуету (Silhouette) характеризує компактність і відокремленість груп у просторі ознак, тоді як F1 та V-measure оцінюють узгодженість з еталонною розміткою. Менша внутрішньокластерна сума квадратів (inertia) у K-Means також не гарантує кращого відтворення тем. Тому ці критерії розглядаються спільно з урахуванням корпусу та налаштувань.

Таблиця 1 – Порівняння засобів для дослідження текстів

Засіб	Корисні можливості	Що необхідно для цієї роботи
scikit-learn	Кластеризація та метрики	Об'єднати з українською обробкою й вимірюванням використання ресурсів
Gensim	Word2Vec, FastText, Doc2Vec	Додати інтерфейс експерименту, оцінювання та звіти
Orange Text Mining	Візуальний аналіз корпусів	Перевірити власну схему Parallel K-Means у керованому сценарії

1.4 Функціональні та нефункціональні вимоги

Відповідно до технічного завдання [1], програма має завантажувати файли у форматах TXT, CSV і JSON, очищати та лематизувати тексти, підтримувати налаштовувані списки стоп-слів і методи TF-IDF, Word2Vec, FastText та Doc2Vec. Для кластеризації передбачено K-Means, агломеративний метод і DBSCAN із параметрами, доступними для налаштування. Програма має обчислювати показники якості й використання ресурсів, будувати візуалізації та експортувати результати у CSV і JSON, а звіти — у PDF або HTML.

До додаткових вимог належать вибір CBOW або Skip-gram, усереднення векторів слів із вагами TF-IDF, метод ліктя, аналіз графіка k-distance, імпорт навчених моделей і збереження міток кластерів для кожного документа. Виконання цих вимог оцінюється окремо від працездатності базового конвеєра. Під час приймання також перевіряють обробку некоректних даних, зрозумілість повідомлень і можливість повторного аналізу збережених результатів.

Технічне завдання (ТЗ) передбачає щонайменше 8 ГБ оперативної пам'яті, 1 ГБ вільного місця на диску та процесор із тактовою частотою від 2 ГГц. Цільовими операційними системами є Windows, Linux і macOS. Ці вимоги потребують перевірки на відповідному обладнанні. Вимога щодо Python 3.8 або новішої версії також потребує уточнення з огляду на використані анотації типів і сумісність залежностей. Фактичне середовище контрольного запуску наведено в розділі 4.

За результатами аналізу доцільно використати бібліотечні алгоритми, зосередивши власну розробку на узгодженні етапів, попередній обробці українських текстів, керуванні експериментами та паралельних незалежних ініціалізаціях. Вимоги до надійності охоплюють збереження початкового корпусу без змін, перевірку формату вхідних даних і збереження результатів кожного завершеного запуску в окремій папці.

2 Проектування програми

2.1 Загальний підхід до проектування

Програму спроектовано як настільний застосунок мовою Python [9] із модульним розподілом відповідальності. Інтерфейс забезпечує введення параметрів і відображення результатів, сервіс керує експериментом, а спеціалізовані модулі виконують обробку даних, векторизацію, кластеризацію та оцінювання. Такий поділ дає змогу перевіряти обчислювальний сценарій без запуску графічного вікна. Послідовність перетворення даних наведено на рис. 1.

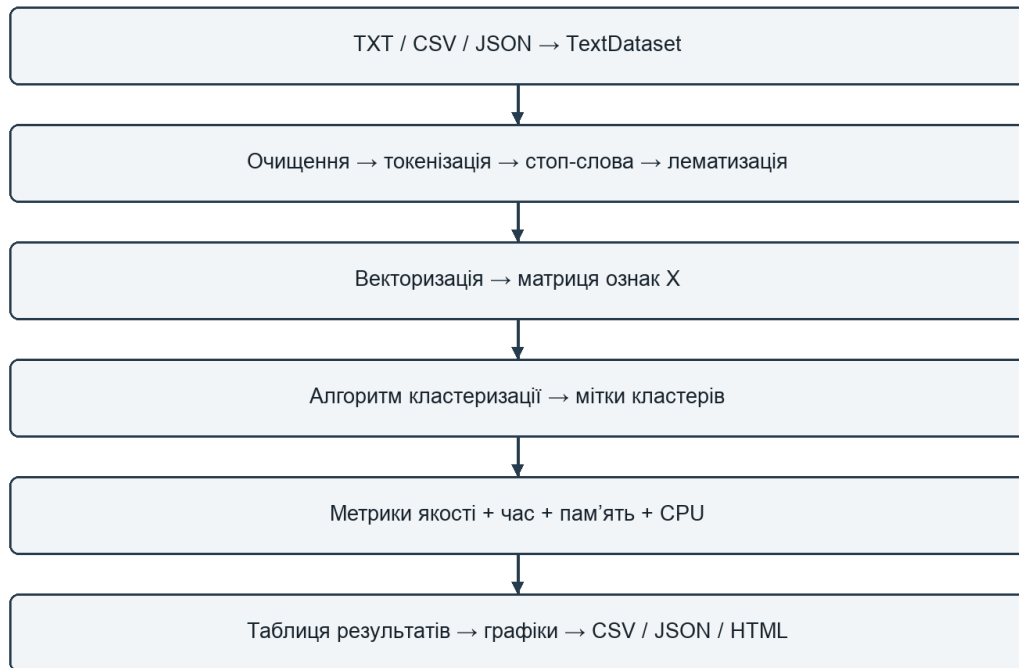


Рисунок 1 – Конвеєр проведення експерименту

Спочатку формується спільний список документів і відповідних наборів tokenів. Далі кожен вибраний векторизатор створює матрицю ознак, на якій послідовно виконуються обрані алгоритми кластеризації. Використання тієї самої матриці дає змогу порівнювати алгоритми за однакового векторного подання корпусу. Еталонні мітки залучаються лише на етапі оцінювання та не впливають на формування кластерів.

Під час проектування застосовано принцип єдиної відповідальності: кожен модуль виконує визначене коло завдань. Адаптери узгоджують бібліотечні моделі зі спільними операціями `fit_transform` і `fit_predict`, зменшуючи залежність керівного коду від конкретного методу. Збереження налаштувань дає змогу простежити умови проведення експерименту. Для підтвердження відповідності окремому стандарту життєвого циклу ПЗ потрібні також регламенти, протоколи перевірок і формальні процедури приймання.

2.2 Зовнішнє проєктування та сценарії користувача

Основним користувачем програми є дослідник, який має базові знання з обробки текстів і машинного навчання. Типовий сценарій починається з вибору файлу та назв стовпців. Далі користувач обирає векторизатори й алгоритми кластеризації, задає параметри та запускає експеримент. Після завершення обчислень він порівнює метрики, переглядає графіки й відкриває звіти. Головне вікно з параметрами контрольного експерименту наведено на рис. 2.

МІСЦЕ ДЛЯ СКРІНШОТА

Вставити скріншот головного вікна Text Clustering Research. Обрати datasets/demo_texts.csv; стовпці text і label; TF-IDF; K-Means, Parallel K-Means, Hierarchical, DBSCAN. Показати K = 5, runs = 3, eps = 0.5, min_samples = 5, 8 перезапусків і 4 потоки. Увімкнути видалення стоп-слів, графіки та рекомендацію K; лематизацію вимкнути. Розгорнути блоки налаштувань; приховати зайві частини робочого столу.

Рисунок 2 – Головне вікно та налаштування експерименту

Кількість кластерів K задається для алгоритмів, які потребують цього параметра. Для DBSCAN визначальними є радіус околу eps і мінімальна кількість точок min_samples, а для агломеративного методу — спосіб об'єднання кластерів linkage. Параметр ініціалізації K-Means визначає спосіб вибору початкових центрів. Поле максимальної кількості текстів обмежує корпус першими N документами; такий відбір не гарантує репрезентативності вибірки.

Інтерфейс побудовано засобами tkinter і ttk. У ньому передбачені згортані секції, таблиця результатів, текстовий висновок, вікно порівняння та перегляд попередніх експериментів. Перегляд збереженого запуску має окрему цінність: користувач отримує доступ до вже обчислених показників без повторного навчання моделей.

Програма працює локально, а результати доступні у графічному інтерфейсі та збережених файлах. Така організація спрощує використання інструмента для індивідуальних досліджень і не потребує окремого вебсервера. Доступ до мережі може знадобитися під час першого завантаження моделі Sentence-BERT, якщо її ще немає в локальному кеші.

Після завершення експерименту користувач має оцінювати показники в сукупності: якість групування, частку шумових документів і витрати ресурсів. Найшвидший метод може не утворити змістовних груп, а найвище значення однієї метрики — не збігатися з найкращою відповідністю еталонним темам. Вікно порівняння підтримує такий аналіз, відображаючи результати кількох комбінацій методів.

2.3 Керування виконанням і обробка помилок

Тривалі обчислення виконуються у фоновому потоці. Метод `_run` формує об'єкт `ExperimentOptions`, запускає `_worker` і планує періодичне опитування черги подій. Сервіс повідомляє про перебіг роботи через функцію зворотного виклику. Події `progress`, `done` та `error` передаються через `queue.Queue`. Метод `_poll`, виконання якого планується викликом `after` з інтервалом 120 мс, оновлює елементи інтерфейсу в головному потоці. Схему взаємодії наведено на рис. 3.



Рисунок 3 – Взаємодія графічного інтерфейсу та фонового обчислення

Черга відокремлює обчислення від оновлення елементів інтерфейсу. Завдяки цьому сервіс не потребує посилань на вікна або таблиці `tkinter`. Метод `run` повертає таблицю `DataFrame` з результатами та словник `artifacts`, який містить шляхи до створених файлів і підсумковий текст. Інтерфейс використовує ці дані для заповнення таблиці, відображення висновку та відкриття графіків.

Під час завантаження перевіряються наявність файлу, його розширення, наявність стовпця з текстами та допустимість обмеження кількості документів. Для запуску потрібні щонайменше два документи; після очищення корпус має містити хоча б один непорожній набір токенів. Помилка окремого алгоритму фіксується в полі `error`, після чого сервіс може перейти до наступної комбінації методів.

Обробка помилок окремих алгоритмів не охоплює весь конвеєр. Створення векторизатора, векторизація та визначення рекомендованого `K` виконуються поза відповідним блоком обробки винятків. Відсутність потрібної бібліотеки або збій на цих етапах можуть припинити поточний запуск. У засобах профілювання також необхідно гарантувати завершення моніторингу через конструкцію `try/finally`. Ці обмеження враховано під час оцінювання надійності.

2.4 Проєктування вдосконаленого підходу

Власний клас `ParallelKMeansClusterer` організовує паралельний запуск кількох незалежних екземплярів K-Means. Для R перезапусків задаються різні початкові значення генератора випадкових чисел. Кожен екземпляр використовує $n_init = 1$ і повертає значення $inertia$ разом із мітками кластерів. Сервіс обирає мітки розв'язку з найменшою сумою квадратів відстаней від точок до центрів їхніх кластерів. Схему алгоритму наведено на рис. 4.

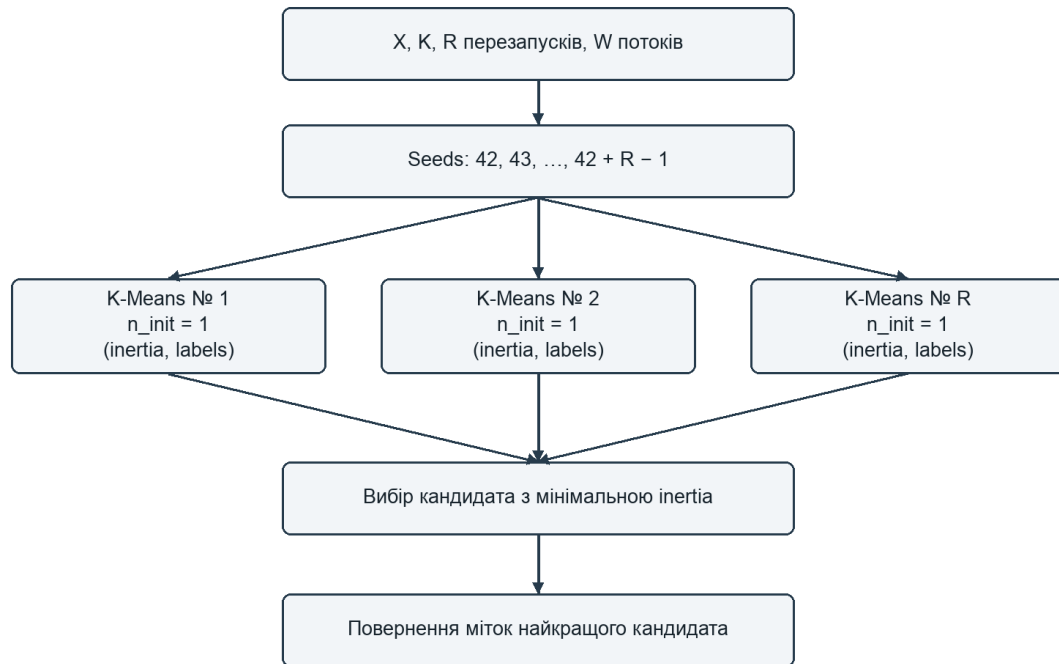


Рисунок 4 – Паралельний багатостартовий K-Means

Пул `ThreadPoolExecutor` використовує не більше $\min(W, R)$ робочих потоків, де W — задана кількість потоків, а R — кількість перезапусків. За замовчуванням $R = 8$, $W = 4$, а початкові значення генератора становлять від 42 до 49. Перед виконанням перевіряються умови $K \geq 2$, $K \leq N$ та додатність R і W , де N — кількість документів. Кожен потік створює власну модель і використовує спільну матрицю X як вхідні дані.

Запропонований підхід змінює організацію багатостартового пошуку, зберігаючи цільову функцію K-Means. Значення $inertia$ вибраного розв'язку є мінімальним серед отриманих кандидатів, проте його зовнішні метрики не обов'язково є найкращими. Для оцінювання прискорення потрібне порівняння з послідовним виконанням тих самих R перезапусків. Порівняння лише з одним запуском змінює також обсяг обчислень. На малому корпусі витрати на керування потоками можуть перевищувати виграш від паралельного виконання.

2.5 Обґрунтування вибору методів і параметрів

Поєднання різних алгоритмів у програмі дає змогу дослідити, як припущення про структуру даних впливають на результат. K-Means формує задану кількість груп навколо центрів, агломеративний метод послідовно об'єднує об'єкти, а DBSCAN виділяє області підвищеної щільності й позначає ізольовані об'єкти як шум. Ці підходи доповнюють один одного під час аналізу того самого корпусу [2].

Для K-Means користувач задає кількість кластерів і спосіб ініціалізації центрів. Кількість кластерів визначає рівень деталізації групування: надто мале значення може об'єднати різні теми, а надто велике — розділити одну тему на кілька груп. Рекомендація за коефіцієнтом силуету допомагає звужити вибір, але її слід зіставляти зі змістом документів. Високе значення внутрішньої метрики саме по собі не встановлює правильну кількість тем.

В агломеративній кластеризації параметр `linkage` визначає правило оцінювання близькості груп. Для `ward` використовується приріст внутрішньокластерної суми квадратів; `single`, `complete` та `average` спираються відповідно на найменшу, найбільшу й середню відстань між об'єктами груп. Через відмінність цих правил однакові вхідні вектори можуть утворювати різні ієрархії. Саме тому обраний спосіб об'єднання потрібно зберігати разом із результатами [2].

У DBSCAN кількість кластерів наперед не задається. Налаштування `eps` і `min_samples` визначають, які околиці вважаються достатньо щільними. Параметри слід добирати для конкретного векторного подання: величини відстаней у TF-IDF та щільних семантичних векторах можуть відрізнятися. Надмірна частка шуму є підставою переглянути налаштування або спосіб подання тексту, а не лише порівняти час виконання.

Щоб відокремити вплив окремих чинників, доцільно змінювати один етап конвеєра за незмінних інших умов. Наприклад, вплив векторизації досліджують з однаковим алгоритмом кластеризації та складом корпусу. Для перевірки лематизації порівнюють два запуски з однаковими методами й параметрами, відмінність між якими полягає лише в її застосуванні. Така схема полегшує пояснення причин зміни метрик.

3 Компонентна структура програми

3.1 Розробка інструментальних засобів для дослідження кластеризації текстів

Компонентна структура відображає фактичні модулі проєкту та основний графічний сценарій. Точка входу `main.py` типово запускає інтерфейс, а параметр `--batch` активує окремий пакетний сценарій. Центральним координатором інтерактивного режиму є `ExperimentService`. Логічні компоненти та напрямки обробки наведено на рис. 5; стрілки від сервісу позначають керування компонентами, а стрілки до сховища — збереження результатів.

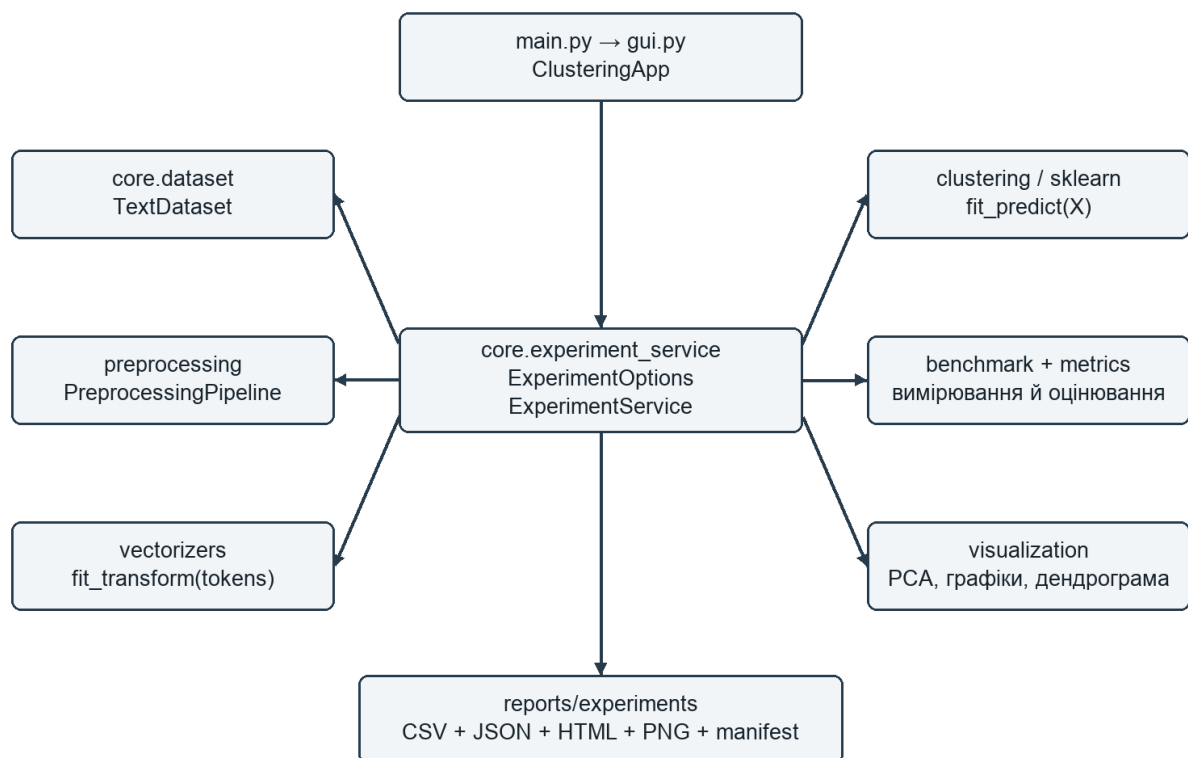


Рисунок 5 – Компонентна структура інтерактивного режиму

Пакетний режим використовує класи `ExperimentRunner`, `Config` та `ExperimentResult`. Він має окрему логіку створення моделей, завчасно імпортує залежності та формує власний звіт. Тому можливості графічного режиму не слід автоматично вважати доступними за запуску з параметром `--batch`. Контрольні випробування зосереджено на `ExperimentService` — сервісі, який використовує графічний застосунок.

Дані між етапами передаються у вигляді списків текстів і токенів, матриці `NumPy`, масиву міток, словників показників та таблиці `DataFrame`. Спільні вимоги до вхідних і вихідних даних зменшують взаємозалежність модулів. Вибір конкретних реалізацій здійснюється у фабричних методах сервісу за назвою методу. Для підключення нового алгоритму потрібно додати відповідну гілку створення моделі та оновити перелік доступних методів.

3.2 Призначення основних компонентів

Таблиця 2 – Призначення компонентів програми

Компонент	Призначення та основні операції
gui.py	ClusteringApp: налаштування, фоновий запуск, черга подій, таблиця, перегляд графіків та історії.
core/dataset.py	TextDataset.load: читання TXT/CSV/JSON; тексти та необов'язкові еталонні мітки.
preprocessing	TextCleaner, SimpleTokenizer, StopwordRemover, Lemmatizer; process(text) → список tokenів.
vectorizers	Адаптери TF-IDF, Word2Vec, FastText, Doc2Vec, Sentence-BERT; fit_transform → матриця ознак.
clustering	Адаптери алгоритмів і ParallelKMeansClusterer; fit_predict → мітки кластерів.
benchmark	Timer, MemoryTracker, CPUMonitor, BenchmarkRunner; результат операції та агреговані вимірювання.
metrics	evaluate_internal та evaluate_external: внутрішні та зовнішні оцінки кластеризації.
visualization	Побудова проєкцій і графіків; у GUI використовується PCA.
core/experiment_service.py	Послідовність експерименту, вибір моделей, рекомендація K, експорт та повторне читання.

3.3 Реалізація обробки та векторизації

Компонент TextCleaner переводить текст у нижній регістр, видаляє URL, цифри та символи, що не належать до дозволеного набору літер і пробілів. SimpleTokenizer розбиває очищений рядок на токени за пробілами. За потреби далі застосовуються базовий і користувацький списки стоп-слів та морфологічний аналізатор rymorphy3 для української мови. Стоп-слова вилучаються до лематизації, тому до власного списку слід додавати ті словоформи, які трапляються у вхідному тексті.

Адаптер TF-IDF обмежує словник 5000 ознаками та перетворює розріджену матрицю на щільний масив. Адаптери Word2Vec і FastText усереднюють вектори слів документа, а за відсутності придатних векторів повертають нульовий вектор. Doc2Vec використовує унікальні ідентифікатори документів. Ці моделі навчаються на поточному корпусі, тому дослід із 50 короткими текстами не дає підстав для загальних висновків про якість їхніх семантичних подань.

Sentence-BERT завантажує paraphrase-multilingual-MiniLM-L12-v2 і кодує рядки, зібрані з оброблених tokenів. Отже, модель отримує вже очищений і, за вибором користувача, лематизований текст. Вплив такого перетворення на контекстні подання потребує окремого порівняння з кодуванням початкових речень.

3.4 Організація даних та збереження результатів

У вхідному файлі TXT кожен непорожній рядок відповідає окремому документу. Для CSV і JSON користувач указує назву поля тексту та, за наявності, поля еталонних міток. Параметр `max_documents` обмежує вибірку першими N записами. Завантажувач перетворює значення текстового поля на рядки, тому пропуски потребують окремої обробки, щоб службові позначення не потрапляли до корпусу. Основні структури даних наведено на рис. 6.

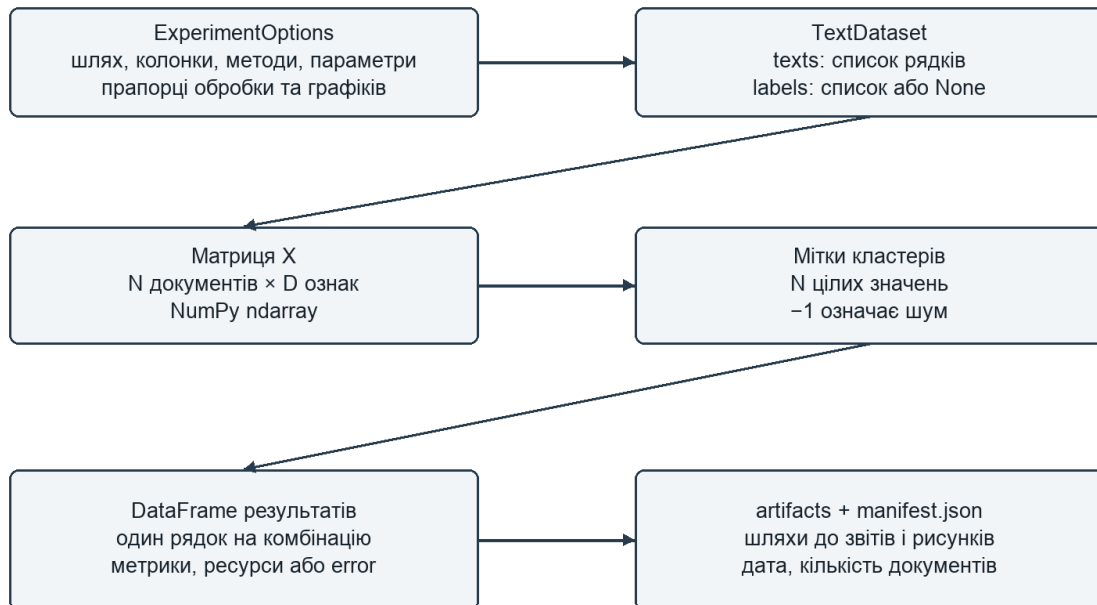


Рисунок 6 – Структури даних та результати експерименту

Рядок результатів містить назву корпусу, назви методів, кількість документів, кластерів і шумових точок, час виконання, показники пам'яті та процесорного навантаження, а також метрики якості. Рекомендоване K визначається окремим перебором значень від 2 до $\min(10, N - 1)$: для кожного значення виконується K-Means і обчислюється Silhouette [8]. Рекомендація не замінює автоматично кількість кластерів, задану користувачем.

Файли групуються в `reports/experiments/experiment_YYYYMMDD_NHMMSS`. CSV зручний для табличного аналізу, JSON — для повторного читання програмою, HTML — для перегляду звіту. Конфігурація серіалізується з `ExperimentOptions` у плоский JSON, а `manifest.json` пов'язує результати з рисунками. Це відрізняється від вкладеного прикладу конфігурації у ТЗ. Поточний експорт містить агреговані показники, але не таблицю відповідності кожного тексту його кластеру.

Окрема папка для кожного запуску зменшує ризик випадкового перезапису результатів, однак не забезпечує резервного копіювання. Для відтворення експерименту надалі доцільно зберігати копію або хеш корпусу, версії залежностей і початкові значення генераторів випадкових чисел. Відносні шляхи полегшать перенесення результатів, а унікальний ідентифікатор запуску усуне ризик збігу часових позначок із точністю до секунди.

3.5 Оцінювання результатів і розширення програми

Компонент оцінювання використовує два джерела інформації: геометрію матриці ознак і, за наявності, еталонні мітки документів. Внутрішні показники доступні без ручної розмітки. Зовнішні показники дозволяють перевірити, наскільки отримане групування відповідає визначеним темам. Відсутність еталонних міток не перешкоджає кластеризації, проте обмежує можливість оцінити її змістову відповідність.

Коефіцієнт силуету зіставляє відстані документа до об'єктів власного кластера та найближчого іншого кластера. Індекс Davies–Bouldin характеризує співвідношення розсіювання всередині груп і відстаней між їхніми центрами. Індекс Calinski–Harabasz зіставляє міжкластерне та внутрішньокластерне розсіювання. За коректно обчислених значень більші Silhouette і Calinski–Harabasz та менший Davies–Bouldin відповідають кращому результату за відповідним критерієм. Ці показники слід інтерпретувати в межах використаного простору ознак [2].

Для F1 застосовано макро- та мікроусереднення. Макроусереднення надає однакову вагу оцінкам класів, тоді як мікроусереднення об'єднує внески окремих об'єктів. V-measure, ARI та NMI доповнюють аналіз узгодженості розбиттів. Номери кластерів не мають самостійного змісту: перестановка номерів не повинна змінювати оцінку якості. Саме тому перед розрахунком F1 реалізовано зіставлення кластерів із класами, описане в підрозділі 4.3.

BenchmarkRunner повторює вимірювану операцію задану кількість разів і обчислює середні значення показників. До наступного етапу передається результат останнього повтору, тому наведені метрики якості характеризують саме цей результат, а не середню якість усіх повторів. Розмежування повторних вимірювань часу та незалежних запусків для оцінювання якості є необхідним для коректного планування дослідження.

Для розширення програми новий векторизатор має приймати список наборів токенів і повертати матрицю з одним рядком на документ. Новий кластеризатор має повертати масив міток такої самої довжини, як кількість рядків матриці. Збереження порядку документів є обов'язковою умовою правильного оцінювання. Після додавання реалізації необхідно оновити фабрику сервісу, перелік методів інтерфейсу та перевірити типовий і помилковий сценарії її використання.

4 Тестування програми

4.1 Методика та середовище перевірки

Перевірку виконано 20 вересня 2026 року в середовищі macOS 27.0 з Python 3.12.13. Використано віртуальне середовище проєкту та демонстраційний корпус datasets/demo_texts.csv, який містить 50 україномовних документів. Додатково перевірено завантаження корпусу у форматах TXT і JSON. Мета перевірки — підтвердити працездатність основного сценарію на малому наборі даних; навантажувальні та кросплатформні випробування до неї не входили.

У репозиторії передбачено два автоматичні тести попередньої обробки: перевірку видалення URL і переведення тексту в нижній регістр. Запуск `python -m pytest tests/ -q` завершився успішним проходженням обох тестів (2 passed). Додатковий інтеграційний сценарій охопив сервісний конвеєр, збереження результатів та їх повторне читання. Перелік виконаних перевірок наведено в таблиці 3.

Таблиця 3 – Фактично виконані перевірки

Перевірка	Очікуваний результат	Фактичний результат
Очищення URL	URL вилучено, слово «сьогодні» збережено	Пройдено
Нижній регістр	«ВЕЛИКІ літери» → «великі», «літери»	Пройдено
Імпорт TXT	50 документів	Пройдено
Імпорт CSV	50 документів	Пройдено
Імпорт JSON	50 документів	Пройдено
Повний цикл без лематизації	4 комбінації, 3 повтори	Пройдено
Відновлення	4 рядки без повторних обчислень	Пройдено
Відсутній файл	FileNotFoundError	Пройдено
Відсутня стовпець	ValueError	Пройдено

Для інтеграційної перевірки обрано TF-IDF і чотири алгоритми: K-Means, Parallel K-Means, агломеративний метод та DBSCAN. Параметри запуску: $K = 5$, $\text{linkage} = \text{ward}$, $\text{eps} = 0.5$, $\text{min_samples} = 5$, $R = 8$, $W = 4$. Увімкнено видалення стоп-слів, побудову графіків і визначення рекомендованого K ; лематизацію вимкнено. Кожну операцію, для якої вимірювали ресурси, виконано тричі. Часові показники є середніми значеннями за трьома повторами.

У межах контрольного сценарію отримано чотири рядки результатів без повідомлень про помилки, перевірено створені файли та повторно завантажено всі чотири записи. Негативні сценарії підтвердили виникнення `FileNotFoundError` за відсутності файлу та `ValueError` за відсутності стовпця з текстами. Ці перевірки не охоплюють ручної взаємодії з графічним інтерфейсом.

4.2 Результати контрольного експерименту

Початковий запуск із лематизацією завершився винятком `ValueError`: бібліотека `rumorphy3` не знайшла українського словника; у середовищі був доступний лише російський. Пакет `rumorphy3-dicts-uk` зазначено у `requirements.txt`, проте в перевіреному середовищі його не встановлено. Подальший контрольний запуск виконано без лематизації. Для підтвердження її працездатності необхідно встановити словник і повторити перевірку.

Таблиця 4 – Результати TF-IDF на демонстраційному корпусі

Алгоритм	Кластери / шум	Silhouette	F1 macro	Час, с
K-Means	5 / 0	0,0026	0,2390	0,0040
Parallel K-Means	5 / 0	0,0069	0,2364	0,0340
Агломеративний	5 / 0	0,0106	0,3029	0,1392
DBSCAN	0 / 50	-1,0000	0,0000	0,0028

Графік середнього часу кластеризації, сформований під час цього запуску, наведено на рис. 7. Час векторизації вимірюється окремо; підготовка корпусу, рекомендація K, завантаження моделі та побудова рисунків не входять до показника `clustering_seconds`. Тому цей показник не дорівнює тривалості всього експерименту.

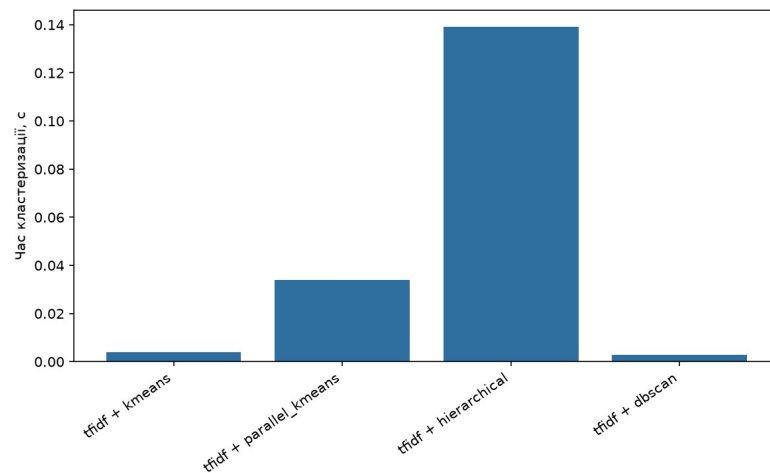


Рисунок 7 – Порівняння часу кластеризації у контрольному запуску

Найвище значення `Silhouette` отримано для агломеративного методу — 0,0106; він також має найбільший `F1 macro` — 0,3029. Водночас близькі до нуля значення `Silhouette` вказують на слабку відокремленість груп у вибраному просторі ознак. `Parallel K-Means` підвищив `Silhouette` порівняно зі звичайним `K-Means` із 0,0026 до 0,0069, проте `F1 macro` дещо знизився — із 0,2390 до 0,2364. Отже, поліпшення одного критерію не означає поліпшення іншого. Три повтори з однаковими початковими умовами характеризують коливання часу, але не стійкість якості за різних ініціалізацій.

Найменший час зафіксовано для `DBSCAN` — 0,0028 с, однак за заданих параметрів усі 50 документів віднесено до шуму. Тому швидкість цього запуску не свідчить про придатність отриманого групування. Значення `-1` у таблиці внутрішніх метрик позначає

неможливість обчислення оцінки за прийнятою в програмі домовленістю. Зокрема, для індексу Davies–Bouldin його не можна інтерпретувати як найкращий результат.

4.3 Візуальна та метрична перевірка

Дендрограму контрольного запуску з параметром `linkage = ward` наведено на рис. 8. Вона відображає послідовність об'єднання об'єктів і значення критерію об'єднання на кожному кроці. Для корпусу з 50 документів програма приховує підписи листків, щоб не перевантажувати рисунок. Дендрограма допомагає дослідити ієрархічну структуру, однак сама по собі не підтверджує відповідності кластерів еталонним темам.

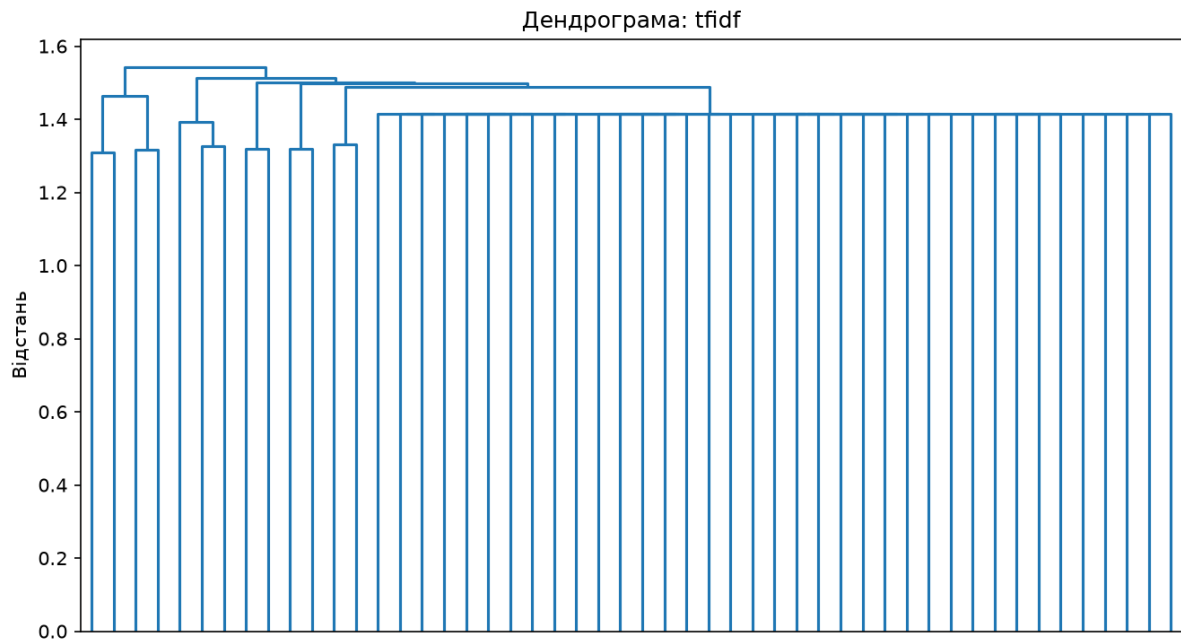


Рисунок 8 – Дендрограма TF-IDF для агломеративного аналізу

У поточній реалізації для побудови дендрограми завжди використовується метод `ward`, навіть якщо для кластеризації обрано інший спосіб об'єднання. У контрольному запуску ці параметри узгоджені. За використання `single`, `complete` або `average` необхідно передавати відповідний параметр і до засобу побудови дендрограми або чітко зазначати, що рисунок відображає результати окремого аналізу.

Перед обчисленням F1 кластери зіставляються з еталонними класами за допомогою `linear_sum_assignment` так, щоб максимізувати кількість збігів. Зіставлення є взаємно однозначним у межах утворених пар. ARI, NMI та V-measure обчислюються за початковими мітками кластерів. Шумові документи перед зовнішнім оцінюванням вилючаються, тому поряд із метриками необхідно аналізувати `noise_documents`. Для внутрішніх метрик шум за наявності кількох кластерів окремо не вилучається, і мітка `-1` може враховуватися як додаткова група.

Засіб `tracemalloc` вимірює пік відстежуваних виділень пам'яті, а не повний обсяг оперативної пам'яті, зайнятої процесом. Показник процесорного навантаження нормується на кількість логічних ядер. Водночас значення для окремих ядер характеризують системне навантаження і можуть включати роботу інших програм. Інтервал моніторингу 0,1 с обмежує інформативність вимірювань коротких операцій. Тому отримані дані слід використовувати як допоміжні ресурсні характеристики.

4.4 Перевірка інтерфейсу та відповідності технічному завданню

Ручна перевірка передбачає відкриття контрольного запуску через кнопку «Попередні експерименти...», зіставлення таблиці з експортованими даними та перегляд вікна порівняння. Приклад відображення результатів наведено на рис. 9. Цей етап має бути завершений після вставлення знімка екрана; результати автоматичних перевірок не підтверджують коректності всіх дій користувача в інтерфейсі.

МІСЦЕ ДЛЯ СКРІНШОТА

Вставити скріншот після відкриття збереженого контрольного запуску з reports/experiments/experiment_20260920_152430. Показати чотири рядки TF-IDF, стовпці алгоритму, Silhouette, часу й пам'яті та загальний висновок. За потреби скористатися кнопкою «Порівняти результати». Числа мають відповідати таблиці 4; скріншот нового запуску матиме інші часові показники.

Рисунок 9 – Відображення результатів та порівняння методів

Таблиця 5 – Стан виконання ключових вимог за аналізом коду

Вимога	Стан поточної версії
TXT/CSV/JSON; базова обробка	Реалізовано; базовий шлях перевірено
TF-IDF; K-Means; ієрархія; DBSCAN	Реалізовано; виконано контрольний запуск
Word2Vec, FastText, Doc2Vec, SBERT	Є реалізації; повний цикл у цій перевірці не запускався
CBOW/Skip-gram; зважене усереднення	Вибір архітектури не передбачено; реалізовано просте усереднення
Автоматичний підбір параметрів	Є рекомендація K за Silhouette; метод ліктя й аналіз k-distance відсутні
Окреме вимірювання навчання та прогнозування	Вимірюється fit_predict загалом
Експорт і збереження моделей	CSV/JSON/HTML є; довільний імпорт моделей і експорт міток документів відсутні
Резервування та кросплатформність	Запуски зберігаються окремо; резервне копіювання та роботу на всіх ОС не перевірено

Подальші випробування мають охопити порожні й однакові тексти, $K \geq N$, пропущені значення, недоступні моделі, помилки запису, великі корпуси та повторюваність результатів стохастичних моделей. Для перевірки прискорення Parallel K-Means потрібне послідовне виконання тих самих восьми перезапусків за однакових початкових значень генератора. Також слід контролювати кількість внутрішніх потоків чисельних бібліотек, щоб зіставляти однакові обчислювальні умови.

5 Висновки

У межах практики розглянуто й перевірено програмний інструмент для дослідження кластеризації природномовних текстів. Аналіз предметної області показав, що оцінювати потрібно весь конвеєр: спосіб попередньої обробки та векторизації визначає простір, у якому працює кластеризатор. Вибір методу лише за одним показником або за результатом одного малого корпусу не дає достатнього обґрунтування для практичного застосування.

Спроектowana модульна структура відокремлює графічний інтерфейс, керування експериментом, завантаження даних, мовну обробку, алгоритми й оцінювання. Зовнішні аспекти проектування охоплюють сценарії налаштування, запуску, порівняння та повторного перегляду. Внутрішні аспекти охоплюють контракти компонентів, передачу матриці ознак і міток, фонове виконання, чергу подій та організацію файлових результатів. Згенеровані схеми відображають ці рішення та пов'язують структуру з фактичним кодом.

Власний Parallel K-Means забезпечує паралельне виконання незалежних перезапусків і вибір розв'язку з мінімальною внутрішньокластерною сумою квадратів. Такий підхід дає змогу керувати кількістю початкових розв'язків і досліджувати вплив кількості потоків. У контрольному досліді вищий Silhouette порівняно зі звичайним K-Means супроводжувався нижчим F1 масо та більшим часом виконання. Отже, отримані результати не підтверджують загальної переваги підходу й потребують перевірки на інших корпусах.

Виявлено відсутність українського словника rymorphy3; контрольний конвеєр перевірено з вимкненою лематизацією. Під час перевірки успішно виконано два наявні модульні тести, імпорт трьох форматів, сервісний запуск чотирьох комбінацій TF-IDF з алгоритмами кластеризації, повторне читання результатів і два негативні сценарії завантаження. Програма сформувала метрики, графіки, конфігурацію та звіти. Це підтверджує працездатність перевіреного базового сценарію, але не є доказом повної відповідності всім вимогам ТЗ або придатності до великих промислових корпусів.

Подальше вдосконалення має включати вибір архітектури Word2Vec, зважену агрегацію, метод ліктя й k-distance, експорт міток документів, узгодження дендрограми з linkage та надійне завершення профілювання після винятків. Для відтворюваності потрібні фіксація версій, хеш корпусу та керування всіма джерелами випадковості. Окремим дослідницьким етапом має стати оцінювання на кількох розмічених україномовних корпусах різних обсягів і тематик.

Практична цінність результату полягає в можливості проводити порівняльні експерименти за узгодженою схемою та зберігати їхні налаштування й показники. Аналіз вимог, опис архітектури та результати перевірки формують основу для подальшого дослідження методів кластеризації україномовних текстів. Наступний етап роботи має поєднувати завершення вимог ТЗ із розширенням експериментальної бази.

Список використаних джерел

1. Дослідження методів кластеризації природомовних текстів. Технічне завдання. Локальний документ проєкту «Технічне завдання.docx».
 2. scikit-learn developers. Clustering. Офіційна документація.
<https://scikit-learn.org/stable/modules/clustering.html>
 3. Gensim. models.word2vec — Word2vec embeddings. Офіційна документація.
<https://radimrehurek.com/gensim/models/word2vec.html>
 4. Bojanowski P., Grave E., Joulin A., Mikolov T. Enriching Word Vectors with Subword Information. Transactions of the Association for Computational Linguistics. 2017. Vol. 5. P. 135–146. <https://aclanthology.org/Q17-1010/>
 5. Le Q., Mikolov T. Distributed Representations of Sentences and Documents. Proceedings of the 31st International Conference on Machine Learning. 2014. Vol. 32. P. 1188–1196.
<https://proceedings.mlr.press/v32/le14.html>
 6. Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP-IJCNLP. 2019. P. 3982–3992. DOI: 10.18653/v1/D19-1410.
<https://aclanthology.org/D19-1410/>
 7. Orange Data Mining. Widget Catalog; Text Mining. Офіційний каталог компонентів.
<https://orangedatamining.com/widget-catalog/>
 8. scikit-learn developers. Selecting the number of clusters with silhouette analysis on KMeans clustering.
https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_silhouette_analysis
 9. Text Clustering Research. Вихідний код проєкту: README.md, gui.py, core, preprocessing, vectorizers, clustering, metrics, benchmark, visualization, tests. Версія робочої папки, перевірена 20.09.2026.
- Дата звернення до електронних джерел: 20.09.2026.